Agentic Synthesis against Counterexample-Supplemented Sketches

Muness Castle Eric Rubeck

July 1, 2026

Abstract

Coding agents can synthesize useful patches while their outputs diverge from unstated policy. A prompt can describe what to build and still leave the disallowed near miss implicit: which plausible implementation would satisfy the visible check while violating the domain rule. We use the term *agentic synthesis against counterexample-supplemented sketches* for a repository-native workflow that makes those near misses part of the specification.

The workflow has five parts. A human writes a partial program-like sketch that fixes strategy and names holes. A finite corpus stores observations and promoted counterexamples. Subject-matter experts correct concrete outputs. A coding agent edits deterministic code and prompt surfaces under known-code anchors. The workflow treats completion as blocked unless a gate reports replay and semantic-comparison success for every promoted case.

The method borrows the loop grammar of sketching and counterexample-guided inductive synthesis (CEGIS) for a different execution surface. Classical sketching and CEGIS keep stronger claims when the search space, specification, and checker can be formalized. Here, the synthesizer is an IDE-shaped coding agent whose edits span ordinary source code and LLM prompt surfaces. The bounded claim is finite-corpus gate satisfaction: when the gate passes, the current strategy satisfies the repository's encoded replay and comparison predicates for every promoted case.

In the authors' proprietary deployment, this loop ran inside an enterprise harness: a custom Cursor extension, `cursor://`-style commands, and an embedded web application through which subject-matter experts loaded production examples, corrected outputs, and helped turn those corrections into input/output specifications. The public companion repository provides a sanitized RosterSynth worked example that implements the paper's disclosed control structure. The paper develops the technique; the repository supplies runnable evidence for the worked example and finite-corpus gate claim.

1 Introduction

A coding agent can produce a plausible patch before the domain rule has been made explicit. That is the risk this paper addresses. Given a task, a codebase, a prompt, and a visible signal to satisfy, an agent may produce a local change that turns the signal green while missing the policy that made a different local change unacceptable.

The human question is more specific than "did the code run?" It is: did this patch preserve the rule the prompt failed to spell out? In policy-heavy workflows, the answer often depends on information that lives outside the code path the agent just edited. A roster remediation may need to choose cancellation before adjustment. A parser may need to treat only the first prefix as status. A data-cleaning pipeline may need to preserve a client-specific exception that looks like an ordinary edge case from the agent's local view.

Conventional tests catch many failures. They can say that output shape is valid, an exception disappeared, or a state delta closed. Unless designed to do so, a test may omit the reason a nearby repair would have been tempting and wrong, the rule the failure exposed, and the path a future

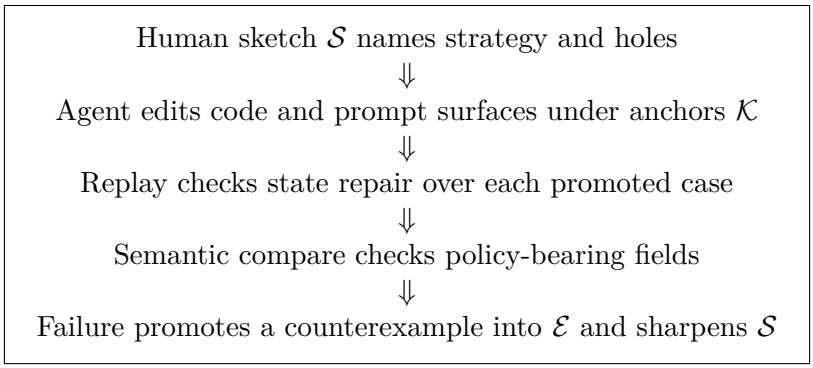

Figure 1: The repository loop that turns a tempting wrong repair into durable guidance.

repair should follow. Natural-language prompts have the complementary weakness. Run-local or unversioned prompts can describe ambiguous intent, but they fork easily and may diverge from the repository's tested behavior. For this class of workflow, the repository needs a stable artifact that records both the human strategy and the examples that sharpen it.

This paper describes that artifact and the loop around it. The ingredients are:

1. a **sketch** $\mathcal{S}$: a partial program in prose and code-shaped structure;
2. a **counterexample-supplemented corpus** $\mathcal{E}$: examples, observations, and promoted failures;
3. **known-code anchors** $\mathcal{K}$: codebase-local shapes, APIs, conventions, and reference paths the agent must preserve;
4. a **dual-oracle implementation surface**: deterministic code for encodable holes and prompt-mediated completion for narrative holes;
5. a **gate** $\mathcal{G}$: replay plus semantic comparison over every promoted case.

The sketch records the human's current strategy: which repairs are permitted, prioritized, or forbidden. The corpus records cases that have earned specification authority through promotion. The gate is the executable boundary for the current done state. A failure becomes valuable when it changes the sketch and the corpus so future agent passes inherit the rule instead of rediscovering the same local trap.

Central claim. We call a coding-agent workflow inspectable when every promoted counterexample connects four things: the sketch clause it supplements, the implementation or prompt surface that handles it, the replay check that verifies state repair, and the semantic compare check that rejects the tempting wrong repair. This is an artifact-level definition of inspectability. It gives maintainers a bounded gate invariant for the current promoted corpus and a promotion procedure for adding new failures.

Positioning. The comparison target is bounded. Existing agentic workflows can make tests and prompts durable, and conventional CEGIS already has counterexample discipline when its formal assumptions hold. This paper addresses a narrower repository setting where three things often live apart: the human strategy, the promoted counterexample, and the executable check that rejects the tempting repair.

Contributions. This paper contributes a formulation and worked artifact.

1. It formulates counterexample-supplemented sketches as a repository-native artifact for agentic coding.
2. It gives a process model for subject-matter expert corrections becoming input/output specs, counterexamples, and golden data.
3. It separates two hole-filling roles: Oracle A, deterministic code the agent maintains, and Oracle B, prompt/LLM/cassette behavior for sketch-declared narrative holes.
4. It states a finite-corpus gate-satisfaction theorem for replay/compare gates and gives the short proof obligation.
5. It gives a sketch-level RosterSynth example and a companion repository for runnable inspection.

The paper proceeds from origin and problem statement through lineage, artifacts, lifecycle, gate semantics, bounded proof, worked example, adoption workflow, companion artifact, discussion, and limitations. The order matters for the argument. The enterprise origin explains why ordinary tests and prompts were insufficient. The lineage shows which pieces come from sketching, CEGIS, programming by example, and agent benchmarks. The formal section then states exactly which predicate a passing gate establishes and which obligations remain human work.

2 Originating setting

The method came from a proprietary enterprise deployment. The work involved heterogeneous operational data, multiple downstream clients with their own rules, and subject-matter experts who held domain knowledge that was only partly captured in code. The harness let those experts operate the counterexample loop through a review interface; agents edited code and prompts under repository-local checks.

The deployed system joined six pieces:

1. a custom Cursor extension for invoking repository actions from the IDE;
2. `cursor://`-style URL commands connecting review actions to local agent workflows;
3. an embedded web application where SMEs loaded production examples, inspected proposed remediations, and supplied corrections;
4. LLM-assisted conversion from SME corrections into input/output specifications;
5. a counterexample loop that promoted selected failures into a golden dataset;
6. non-developer operation of the loop for code and prompt implementations of a complex data-cleansing pipeline.

Those details matter because they fix the intended problem scale. The method targets teams where the domain rule lives with an expert, the implementation lives in code and prompts, and correctness requires a durable bridge between them. In that setting, an expert correction is more than feedback on a single row. It is a candidate specification change that needs a path into the repository.

The private harness itself remains outside the public artifact. Production data, proprietary extension code, client-specific rules, and private SME workflows are unavailable for public inspection. We therefore use the enterprise setting as author-reported motivation and use the public companion repository as executable evidence for the disclosed control structure. The reusable loop is:

$$\begin{aligned}\text{SME correction} &\rightarrow \text{input/output spec} \rightarrow \text{promoted counterexample} \\ &\rightarrow \text{sketch update} \rightarrow \text{agent repair} \rightarrow \text{replay/compare gate}\end{aligned}$$

The public repository strips the setting down to a sanitized example: a paper, fixtures, tests, provenance, recorded model outputs, and a small roster remediation case. That example does not validate the private deployment. It gives readers a concrete artifact for inspecting how the method represents policy, encodes a promoted counterexample, and checks a repaired strategy.

3 Problem

Coding agents alter the economics of software change. They reduce the cost of producing a candidate patch. They also increase the number of candidate patches that look acceptable under incomplete specifications. That combination makes the specification boundary more important than the synthesis engine. Cheap patch generation helps only when the surrounding artifacts say which patch should be generated.

The failure mode has a recognizable shape:

1. A user gives an agent a domain task.
2. The prompt names the obvious state gap.
3. The agent finds a local patch that closes the visible gap.
4. A subject-matter expert recognizes a policy violation.
5. The violation was present in the domain, yet absent from the checkable artifacts.

The patch can be technically clean. The agent can follow local style. The tests can pass. The missing piece is the promoted counterexample: the artifact that says, "this plausible repair is forbidden for this reason, and future repairs must satisfy this rule." Without that artifact, the team has learned something in conversation while leaving the repository nearly unchanged.

3.1 Tests need intent memory

A test is a predicate over behavior. A counterexample-supplemented sketch adds a memory of intent. The distinction matters when several outputs satisfy a visible predicate. A replay check can show that a proposed row closes a state delta. A semantic compare check can show that the row used the expected operation, target, date, or status. The sketch explains why those semantic fields matter.

The issue is the gap between what many tests encode and what a policy-heavy repair needs to preserve. Tests work well when the expected behavior is already explicit. A counterexample-supplemented sketch helps at the moment when the failure exposes a rule that had been implicit: it gives the rule a name, links it to the fixture that exposed it, and tells the next repair which tempting path to avoid.

Tests can also lose policy context when the failure is not linked to the rule it protects. A future maintainer or agent run may see only the failing assertion and patch around it. A sketch clause linked to the test records the policy ordering a later repair should preserve.

3.2 Prompts need repository anchoring

Run-local prompts can carry ambiguous intent in a way formal checks often cannot. They can state business rules, narrative criteria, exception handling, and domain language. That strength becomes fragile when the prompt lives in a chat transcript, notebook, model call, or agent session outside the repository's review path.

The method still uses prompts. It treats them as one implementation surface. Oracle B is prompt-mediated completion constrained by the same sketch and corpus as deterministic code. Prompt text can encode narrative criteria; the resulting output remains subject to the same gate. The repository becomes the place where the prompt, the fixture, the sketch clause, and the check meet.

3.3 A repository-native version of the loop

Formal synthesis and CEGIS already cover a large design space. When the program template, the specification, and the checker can be written in a formal language, solver-backed synthesis can search a constrained space and return results with stronger semantic force than the repository gate in this paper. Those systems also have well-known variations: counterexamples may come from a verifier, examples may guide inductive search, and oracles may help steer component selection or specification refinement [8, 3].

The setting here gives up some of that formal strength in exchange for a different kind of coverage. Enterprise data-cleaning work often mixes deterministic constraints with prose conventions, client-specific policies, ambiguous notes, historical exceptions, and output formats. Some rules can become deterministic code. Others remain prompt-mediated narrative criteria until the team has enough counterexamples to encode them. The method therefore targets a repository-native loop: generate or edit a candidate, run the current gate, promote a counterexample when the gate exposes missing policy, and repair the sketch, code, prompt, or checker that carried the gap.

That choice trades formal strength for operational traceability. The method inherits the CEGIS habit of using counterexamples to constrain the next candidate, while accepting a weaker result: finite-corpus predicate satisfaction under the repository's current semantics. Its value is the explicit join across artifacts that often sit outside one formal specification: comments, prompts, cassettes, fixture names, code anchors, and generated provenance.

4 Lineage

4.1 Sketch and CEGIS

Solar-Lezama's sketching work framed synthesis as collaboration between programmer insight and automated search: the programmer writes a partial program, leaving holes for a synthesizer to fill [6, 7]. Earlier sketching work already used counterexamples to refine candidates in finite-program synthesis settings [8]. The programmer supplies structure; the machine searches within it. CEGIS adds the validation loop: generate a candidate, validate it externally, and feed counterexamples back into the next synthesis step.

That loop carries an important discipline. A counterexample is more than a failing test case. It removes a region of the candidate space by showing a concrete input on which the current candidate violates the specification. In solver-backed settings, the synthesizer can use the growing counterexample set to search a formal space more intelligently than blind generate and check. Oracle-guided synthesis extends the picture by letting an oracle guide or validate choices in component-based synthesis [3].

4.2 Programming by example

Programming-by-example systems such as FlashMeta and PROSE synthesize DSL programs from examples using domain-specific deduction [5, 2]. Those systems show a different but related lesson: examples can be a specification surface when the target language and domain-specific operators are chosen carefully. The examples serve as constraints on a search procedure inside a language designed for that family of tasks.

4.3 Agentic coding and tests

Agentic coding work changes the repair surface again. Agent benchmarks such as SWE-bench evaluate whether language models can resolve real repository issues [4]. Tests-as-prompts work studies tests as both input and evaluation for LLM code generation [1]. These approaches already use repository artifacts and executable checks to steer agents. The method here is compatible with that direction. It adds a specific artifact boundary for policy-heavy repairs: every promoted counterexample should say which sketch clause changed, which tempting repair is rejected, which code or prompt surface handled the rule, and which gate check enforces it.

4.4 Positioning boundary

Agentic synthesis against counterexample-supplemented sketches therefore borrows from several traditions and treats them as live comparators. The sketch resembles a partial program. The corpus resembles the growing example set in CEGIS and programming by example. The gate resembles a bounded validator. The synthesizer is a coding agent with permission to edit source and prompts. The paper's specific claim begins where those pieces meet in an ordinary repository.

The difference is the shape of the hole. In classical Sketch, the hole is usually inside a formal program and the solver searches a formal space. In a repository with code, fixtures, prompts, cassettes, review comments, and generated evidence, the hole can be split across artifacts. A rule may need one deterministic branch, one prompt instruction, one fixture, and one semantic compare field. The method in this paper treats those artifacts as the synthesis surface and uses counterexamples to keep the surface aligned.

	Sketch / CEGIS	Programming by example	This work
Human input	Partial program with holes	Examples inside a DSL	Sketch, SME corrections, counterexamples, anchors
Synthesizer	Solver-backed search	DSL learner	Coding agent editing code and prompts
Failure signal	Counterexample from validator	Missing or inconsistent example	Gate failure or SME correction
Validation	Formal or bounded decision procedure	Example consistency	Replay plus semantic compare over $\mathcal{E}$
Result boundary	Solver-relative result	DSL/example consistency	Finite-corpus predicate satisfaction under repo semantics

Read this table as a positioning map rather than a complete taxonomy of synthesis or agentic coding approaches. Each column stands for a family resemblance that matters to this paper: partial programs, example-driven constraints, repository repair, and executable validation. The method's contribution is the artifact join between those resemblances: how a repository links sketch clauses, promoted counterexamples, code or prompt surfaces, and gate checks.

Artifact	Reader question	Failure if blurred
Sketch	What rule should future repairs preserve?	The prompt drifts and the rule stays hidden.
Corpus	Which cases have specification authority?	Examples become anecdotes, not promoted constraints.
Anchors	What local code shape must the agent preserve?	The agent invents a second convention.
Implementation	Which code or prompt handles the hole?	A paper rule cannot be traced to the surface that applies it.
Gate	What behavior is executable now?	The claim floats above the repository instead of being checked.

Table 1: Artifact responsibilities from a reader's point of view.

5 Artifacts

The method assigns each artifact one job. The sketch records strategy. The corpus records promoted cases. Known-code anchors record local implementation shape. The gate records the current executable semantics. Keeping those jobs separate makes failures easier to interpret: when a gate fails, the maintainer can ask whether the sketch, corpus, anchor, implementation, or checker needs to change.

Definition 1 (Sketch)**.** *A sketch $\mathcal{S}$ is a human-editable artifact that fixes the permitted strategy space and names holes. It may contain prose, tables, pseudo-code, type shapes, policy order, abstention rules, and examples of forbidden repairs. It persists in the repository and changes when a counterexample reveals missing policy.*

A sketch should answer these questions for future maintainers and agent runs:

1. Which operations exist?
2. Which operation has priority when several repairs close the same state gap?
3. Which fields are policy-bearing?
4. Which decisions belong in deterministic code?
5. Which decisions belong in prompt-mediated narrative completion?
6. Which cases force abstention or escalation?

The sketch is deliberately less formal than a full specification. Its job is to hold the policy shape before every detail can be encoded. That incompleteness is useful when the team knows a rule exists but has not yet decided whether it belongs in code, prompt text, fixture data, or human review.

Definition 2 (Corpus)**.** *The corpus $\mathcal{E} = \{(x_i, y_i)\}_{i=1}^{n}$ is the finite set of promoted examples the current gate must satisfy. Each input x_i is a concrete scenario or fixture. Each expected output y_i is the semantic repair the gate enforces.*

The corpus is authoritative because promotion is deliberate. Raw examples can be noisy. SME corrections can contain ambiguity. A promoted counterexample should meet an explicit maintainer-reviewed threshold: it names a plausible wrong output, states the violated rule, and is encoded in the gate. Promotion turns a case from observed behavior into specification pressure.

Definition 3 (Known-code anchor)**.** *A known-code anchor $\mathcal{K}$ is an existing source artifact that constrains the agent's implementation style: an API, result type, parser convention, error shape, fixture format, test helper, reference implementation, or prompt structure. Anchors reduce the risk of parallel local inventions.*

Anchors matter because agentic repair can accidentally create a second convention beside the first. A sketch names the policy. Anchors name the shape the codebase already uses to carry that policy. They are especially important when an agent can satisfy a local test with a new result shape that no downstream caller expects.

Definition 4 (Gate)**.** *A gate $\mathcal{G}$ is the executable validation bundle that evaluates a candidate strategy $\mathcal{H}$ over every promoted case in $\mathcal{E}$. In this method the gate has two layers: replay and semantic compare.*

Replay checks whether the proposed output changes the world state according to the encoded state predicate. Semantic compare checks whether the output used the encoded policy-bearing fields from the promoted expectation. The two layers separate state repair from policy repair. That separation matters because the most dangerous repair can be the one that makes the arithmetic look right while choosing the wrong operation.

6 Roles

The method assigns distinct jobs to humans, agents, and checks. The separation is practical. SMEs know which output is acceptable. Maintainers decide which corrections become specification. Agents repair implementation surfaces. Gates report whether the current strategy satisfies the encoded corpus.

6.1 Subject-matter expert

The SME supplies corrections on concrete examples through a review interface. Their review action says: this proposed output is wrong; the correct output is this; the reason is this domain rule. In the originating harness, the system helped convert that correction into an input/output spec. In the general method, the workflow needs some conversion path from SME judgment to repository artifact.

The SME's correction has three parts:

1. the observed input;
2. the corrected output;
3. the explanation that distinguishes the corrected output from the tempting wrong one.

The explanation matters because it seeds the sketch update and directs repair toward the general rule the fixture exemplifies. A corrected row alone can invite memorization. A corrected row plus a reason tells the next agent why the row matters.

6.2 Developer or maintainer

The developer curates promotion. They decide which SME corrections join $\mathcal{E}$, which sketch clauses change, and which checks enforce the new rule. They also preserve known-code anchors so the agent keeps the existing output shape.

This role is deliberately conservative. Promotion changes the specification, so it requires ownership. A mature harness can assist with extraction, candidate tests, and provenance, but the maintainer still decides whether the promoted case distinguishes a rule, encodes the right fields, and belongs in the gate.

6.3 Coding agent

The agent repairs the artifact under constraints. It may edit deterministic code, prompt text, fixtures, or tests. Its job is synthesis inside the boundary set by $\mathcal{S}$, $\mathcal{E}$, $\mathcal{K}$, and $\mathcal{G}$.

The agent should receive three forms of context before editing:

1. the sketch clause involved;
2. the counterexample and tempting wrong patch;
3. the known-code anchors that define output shape and local conventions.

That context changes the repair problem. The agent is no longer asked only to make a failing signal green. It is asked to preserve a specific rule, reject a known tempting patch, and keep the codebase's existing shape.

6.4 Oracle A: deterministic code

Oracle A handles encodable policy. It should be deterministic, reviewable source code: grouping, filtering, ordering, type construction, deterministic abstention, and domain calculations. The agent can maintain it because the rule has been made concrete enough to encode.

6.5 Oracle B: prompt-mediated completion

Oracle B handles narrative or under-encoded policy: ambiguous notes, human descriptions, client-specific language, or judgment that belongs in a model prompt for the current stage of the system. Oracle B remains constrained by the sketch and checked by the same gate. A cassette can freeze its output for continuous integration paths that depend on model output.

6.6 Hybrid resolver

A hybrid resolver tries Oracle A first, then routes abstentions to Oracle B. This keeps deterministic policy on the reliable path and leaves narrative completion explicit. The split also gives the team a migration path: a rule can begin in prompt-mediated completion, then move into deterministic code once enough counterexamples make it precise.

7 Counterexample lifecycle

The counterexample lifecycle is the core process. It turns an observed failure into a repository artifact the next agent must respect. Each stage has a separate owner so the loop keeps SME judgment, maintainer promotion, agent repair, and executable validation distinct.

7.1 Observation

A concrete case enters the review surface. It can come from production data, a synthetic fixture, a regression report, or a reviewer. The system proposes an output. The SME accepts or corrects it. At this stage the case is evidence, not yet specification.

7.2 Correction

A correction names the desired output and the rule behind it. The correction should also name the tempting wrong repair when possible. This is the moment where tacit domain knowledge becomes explicit: an expert says which nearby output would have fooled the visible check and why the policy rejects it.

7.3 Spec extraction

The harness converts the correction into an input/output spec. An LLM can help draft this spec, especially when the SME explanation is prose. The maintainer decides whether that draft is precise enough to promote; the gate later enforces the encoded expectation. This step is a translation step, not an automatic source of truth.

7.4 Promotion

Promotion makes the case part of $\mathcal{E}$. Promotion is a specification change. The promoted case must contain enough information to reject the tempting wrong repair. A case that only records the correct answer can still leave the failure mode ambiguous.

Promotion also creates maintenance work. The sketch may need a new clause, the comparer may need a new field, the fixture may need a clearer name, and the failure message may need to tell the next agent where to look. If those pieces are missing, the corpus grows without making the repair problem clearer.

7.5 Sketch revision

The sketch changes with the corpus. The revised sketch should state the general rule that the counterexample exposed and the fixture answer that demonstrates it. This protects the next agent pass from memorizing one row. The sketch is the place where the team names the rule; the fixture is the place where the team records the concrete pressure that forced the rule into view.

7.6 Repair

The agent repairs Oracle A, Oracle B, fixtures, or tests under the revised sketch. Known-code anchors constrain result shape and local convention. The repair should close the promoted case while preserving earlier promoted cases. Otherwise the loop has only moved the failure.

7.7 Gate

The gate runs replay and semantic compare over all promoted cases. A passing gate establishes the Section 9 result for the current strategy, corpus, checkers, fixtures, cassettes, and golden rows. If any of those change, the gate must run again.

Candidate behavior	Replay	Compare	Interpretation
Open state gap	reject	not reached	Candidate fails encoded state repair.
Closed state, wrong op	accept	reject	Candidate repairs state but violates a policy field.
Closed state, right op	accept	accept	Case satisfies $\mathcal{E}$ under current repo semantics.

Table 2: Replay and semantic compare answer different reader questions.

8 Gate semantics

The gate is the executable semantics of the current corpus. It has two layers because state repair and semantic repair fail differently. Keeping those layers separate gives the next agent a better error signal: a replay failure says the output failed the encoded state repair; a compare failure says the output repaired state while violating an encoded policy field.

Definition 5 (Replay)**.** *Replay applies a candidate output r to an input state x and checks whether the proposed change closes the domain gap. Replay is domain-specific: it may compute balances, apply updates, recalculate derived fields, or simulate downstream effects.*

Definition 6 (Semantic compare)**.** *Semantic compare checks fields that replay leaves open: operation kind, selected entity, work date, issue type, status mutation, route, priority, or any other policy-bearing field encoded by the comparer. A row is compare-valid when it matches the promoted expectation on those fields.*

Definition 7 (Gate pass)**.** *For a strategy $\mathcal{H}$ and corpus $\mathcal{E}$, the gate passes iff every $(x_i, y_i) \in \mathcal{E}$ yields $r_i = \mathcal{H}(x_i)$ such that replay accepts (x_i, r_i) and semantic compare accepts (y_i, r_i).*

Replay is responsible for encoded state-repair failures. Compare is responsible for encoded policy-field violations. A useful gate usually needs both because the two checks reject different wrong repairs. In the roster example, an append repair can close the hours math while choosing the wrong operation; replay accepts the state change and compare rejects the policy violation.

8.1 Design rules for gates

A gate should be deterministic, local, and specific. It should run locally, report the exact promoted case that failed, and preserve the difference between replay failure and compare failure. The failure message should tell the next agent which rule to inspect.

Good gates have these properties:

1. **Promoted corpus coverage.** Every promoted case runs.
2. **Failure specificity.** Replay and compare failures are reported separately.
3. **Stable fixtures.** Live model calls are replaced with cassettes or recorded outputs for CI paths that depend on model output.
4. **Source links.** Each failure can be traced to sketch clause, scenario, and check.

5. **Tempting-patch rejection.** The gate includes at least one check that fails the previously plausible wrong repair.

These rules are design constraints rather than formal assurances. A gate that omits a field cannot enforce that field. A comparer with a bug can certify the wrong behavior. The method therefore treats gate design as specification work and puts checker quality in the limitations rather than hiding it behind the word correctness.

9 Finite-corpus correctness

The proof obligation is intentionally bounded. A passing gate establishes $\mathcal{E}$- correctness relative to the current replay function, semantic comparer, fixtures, cassettes, and golden rows. When a new failure is promoted into $\mathcal{E}$, it creates a larger proof obligation; after the gate passes again, the same bounded result applies to the expanded corpus.

Definition 8 ($\mathcal{E}$-correctness)**.** *A strategy $\mathcal{H}$ is $\mathcal{E}$-correct with respect to replay function R and compare predicate C when, for every $(x_i, y_i) \in \mathcal{E}$, $R(x_i, \mathcal{H}(x_i)) = \text{true}$ and $C(y_i, \mathcal{H}(x_i)) = \text{true}$.*

Lemma 1 (Replay soundness relative to the implementation)**.** *If replay returns true for (x, r), then r satisfies the state-repair condition encoded by replay for x.*

Proof. Replay is the executable definition of state repair for the gate. It evaluates the candidate output against the input state and returns true exactly when the encoded state-repair predicate holds. Therefore a true replay result entails the state-repair condition the gate implements. □

Lemma 2 (Compare soundness relative to the corpus)**.** *If semantic compare returns true for expected output y and candidate output r, then r agrees with y on every policy-bearing field encoded by the comparer.*

Proof. The comparer enumerates the checked policy-bearing fields and fails on mismatch. Therefore a true compare result means every encoded policy-bearing field matched the promoted expectation. □

Theorem 1 (Finite-corpus gate soundness)**.** *If $\mathcal{G}$ passes for strategy $\mathcal{H}$ on corpus $\mathcal{E}$, then $\mathcal{H}$ is $\mathcal{E}$-correct with respect to the replay and compare semantics encoded by the repository.*

Proof. By definition, $\mathcal{G}$ iterates over every $(x_i, y_i) \in \mathcal{E}$, computes $r_i = \mathcal{H}(x_i)$, and requires replay and compare to pass. By replay soundness, each accepted r_i satisfies the state-repair condition encoded by replay for x_i. By compare soundness, each accepted r_i agrees with y_i on every encoded policy-bearing field. Thus every promoted corpus case satisfies both predicates, which is exactly $\mathcal{E}$-correctness. □

Scope. The theorem covers the promoted corpus, the encoded replay checker, the encoded semantic comparer, and the current golden rows. It says nothing about unpromoted failures, unencoded policy fields, future model behavior, or private deployment behavior outside the public artifact. Checker bugs require checker fixes. Wrong golden rows require human correction. The result is narrow enough to audit and strong enough to guide agentic repair: a maintainer can see which predicate passed, which corpus it covered, and which boundary remains open.

Repair	Visible state effect	Policy meaning	Gate result
Append adjustment	hours delta closed	gap compensation	replay accepts; compare rejects
Cancel duplicate	hours delta closed	duplicate removed	replay accepts; compare accepts

Table 3: The worked example separates state repair from policy repair in one small case.

10 Worked example at sketch level

RosterSynth is the paper's sketch-level worked example. It illustrates the loop through one roster remediation case. The companion repository contains the implementation details for running the example, including source, fixtures, tests, recorded model outputs, and generated provenance. The paper stays at sketch level so the reader can see the method before tracing file paths.

10.1 Scenario

A subject-matter expert sees a roster gap and two active bookings that explain it. The visible arithmetic suggests an adjustment. The policy points to duplicate cancellation. The important feature of the example is the ambiguity: two repairs can close the same apparent state gap, and only one follows the rule.

The tempting local interpretation is simple: if badge hours and scheduled hours disagree, append an adjustment that closes the delta. That patch can make the numbers balance. The policy interpretation is different: when duplicate active bookings with the same relevant shape explain the gap, cancel the selected duplicate. The difference is small in output shape and large in meaning.

10.2 Tempting repair

The tempting repair adds an adjustment that closes the hours balance. Replay accepts the state change because the arithmetic gap disappears. Semantic compare rejects the repair because the selected operation is wrong. The method retains this failure because the patch succeeds under one visible predicate while violating the policy-bearing field.

10.3 Sketch rule

The counterexample promotes a sketch rule: duplicate active bookings with the same relevant shape on the pay-window end date cancel the selected duplicate when that cancellation closes the gap. The sketch now tells the next agent which operation has priority over the adjustment. It also tells the maintainer what future changes must preserve: the operation order, the selected duplicate, and the fields that distinguish cancellation from compensation.

10.4 Gate lesson

The example illustrates the paper's central gate lesson in one roster remediation case. Replay asks, "did the proposed output repair the encoded state gap?" Compare asks, "did it use the encoded rule the sketch requires?" The method needs both questions because a state-valid repair can still be a policy violation.

10.5 Artifact boundary

The paper stops at sketch level. The companion repository supplies source, fixtures, tests, recorded model outputs, and provenance for concrete-path inspection. This split lets the paper focus on the technique while the repository carries implementation evidence. A reader who wants the exact command path can follow the README; a reader who wants the claim boundary can stay with the theorem and the sketch-level example.

11 Applying the method

A lightweight version of the workflow can precede a full enterprise harness. The practical objective is to make every agent repair answer three questions: which rule is being used, which example exposed it, and which gate would reject the tempting wrong repair. Teams can start with ordinary repository files and add automation only where the loop already works.

11.1 Start with the sketch

Write the intended strategy before asking the agent for code. The sketch should name the available operations, the priority order, the known holes, and the abstention cases. Even an incomplete sketch can be useful if it gives later failures a place to attach. The first sketch is a working surface, not a claim that the policy is complete.

11.2 Collect examples before polishing code

Examples are useful when they distinguish competing repairs. A happy-path example shows what success looks like. A counterexample shows which plausible success should be rejected. The second kind is especially useful for agentic repair because it teaches the boundary between "green" and "right under this rule."

11.3 Name known-code anchors

Point the agent at existing result types, parser behavior, fixture format, error shape, and prompt conventions. This reduces the chance that the agent creates a second local convention just to satisfy the current case. Anchors turn style and shape from tacit repository knowledge into explicit repair context.

11.4 Separate replay from compare

Build replay and compare as separate checks. A single assertion that says "the output looks right" hides the distinction between state repair and policy repair. Separate failures teach the next agent more: replay failure means the state predicate stayed broken; compare failure means the state predicate passed while a policy-bearing field diverged from the promoted expectation.

11.5 Promote failures deliberately

Every failure should be triaged. Some failures expose fixture bugs. Some expose checker bugs. Some expose missing policy. The last category belongs in $\mathcal{E}$ and $\mathcal{S}$. Promotion is the moment where a case becomes part of the specification.

Deliberate promotion asks for more than a corrected answer. It asks what tempting patch the case rejects, which sketch clause should change, which field the comparer must check, and which failure message will guide a future repair. That extra work is the price of turning a single failure into reusable specification pressure.

11.6 Keep generated evidence subordinate

Graphs, node files, cassettes, and extracted papers help readers inspect evidence. They support the claim after the technique is clear. The main paper should lead with the method, the theorem, and the example at sketch level; generated artifacts then answer provenance and reproducibility questions for readers who want to audit the path.

12 Companion artifact

The companion repository provides an example-first entry point. It contains a sanitized RosterSynth slice, generated provenance, recorded model outputs, and tests that exercise the sketch-level story. The repository README gives the run commands and file map. The paper keeps implementation details in the companion artifact so the main argument stays portable across domains.

The artifact has three jobs:

1. let developers run the example and inspect the gate fail or pass;
2. let academics inspect how the finite-corpus claim maps to code;
3. show future users what would need to change to adapt the workflow to another domain.

The artifact also demonstrates a practical documentation boundary. The main paper explains the technique. The README routes readers. Example docs show the concrete flow. Generated appendices expose provenance for readers who want to audit the evidence. Keeping those jobs separate prevents the implementation appendix from becoming the argument and prevents the paper from turning into a command transcript.

13 Discussion

13.1 Tests name behavior; sketches name intent

Tests can verify behavior. Counterexample-supplemented sketches add the rule the failure exposed. The failing case joins $\mathcal{E}$, and the sketch records what future agent passes must preserve. The useful distinction is temporal: a test tells the current run what passed or failed; a sketch clause tells the next repair why the failure mattered.

13.2 Prompts implement one surface of the contract

The method uses prompts for narrative holes, ambiguous notes, and policy text awaiting full encoding. The sketch and corpus constrain those prompts. The gate checks their outputs. Prompting becomes one implementation surface inside the larger contract, alongside ordinary source code and fixtures.

13.3 SME corrections become specification inputs

In the originating harness, SMEs could submit corrections through the review interface rather than routing every specification change through developers first. The harness converted those corrections into specs and candidate counterexamples, moving domain knowledge closer to the artifact that constrained future agent runs. Maintainers still curated promotion; SME input created the pressure that made the missing rule visible.

13.4 Promotion quality is a scaling constraint

Promotion quality is a central scaling constraint. A large pile of examples gives weak guidance when the examples leave rules ambiguous. A smaller corpus of sharp counterexamples may be more useful than a larger ambiguous corpus because each case rejects a specific tempting patch. Promotion should therefore ask: what tempting patch does this case reject?

13.5 Generated provenance helps after the claim is clear

Graph extraction and node files help audit the path from sketch to check. They remain supporting evidence. The paper should state the method and claim boundary before relying on generated provenance as supporting evidence. Provenance then answers the follow-up question: where is this claim grounded?

14 Limitations

1. **Finite corpus.** The theorem covers $\mathcal{E}$. New failures must be promoted and the gate must pass again before the bounded result covers them. The method does not claim universal correctness across unseen cases.

2. **Checker quality.** Replay and compare inherit the limits of their encoded semantics. A buggy checker can certify the wrong behavior until a maintainer repairs the checker and replays the promoted corpus.

3. **Golden quality.** A wrong golden row makes the gate enforce the wrong behavior. Promotion therefore needs review by someone who understands the domain rule and can distinguish a corrected answer from a corrected specification.

4. **Promotion quality.** A promoted case should distinguish a rule from a recorded answer. Weak promotion gives future agents examples to memorize; strong promotion gives them rules to preserve.

5. **Sketch quality.** Hidden prose rules still require human review. The method gives those rules a repository home, then relies on maintainers to keep that home current as new counterexamples arrive.

6. **Prompt drift.** Live model behavior can drift. Cassettes make model-output-dependent CI paths reproducible; live behavior still needs review before a team trusts a new model response.

7. **Enterprise evidence boundary.** The public companion demonstrates the control structure on sanitized fixtures. The proprietary harness motivates the method and remains outside the public artifact; the public evidence supports the loop and states the enterprise deployment boundary.

These limitations are part of the method rather than cleanup after it. They tell a maintainer where the proof stops and where judgment starts: promotion, checker design, golden review, prompt review, and private deployment claims remain human responsibilities.

15 Conclusion

Agentic synthesis against counterexample-supplemented sketches treats a coding agent as a synthesizer inside a constraint environment. The human writes the sketch, curates the corpus, and promotes failures. The SME corrects concrete outputs. The agent edits code and prompts. The gate supplies the finite-corpus predicate boundary.

The method turns agentic coding from a chat-centered activity into a repository-centered loop: sketch, example, correction, promotion, repair, replay, compare. That loop came from an enterprise setting where SMEs had the domain knowledge and agents had the editing surface. The reusable technique is the bridge between them. Each promoted counterexample adds an executable check against a known wrong repair, and each passing gate states the bounded result: the current strategy satisfies the encoded replay and semantic-comparison predicates for the current promoted corpus.

References

[1] Yi Cui. Tests as prompt: A test-driven-development benchmark for llm code generation, 2025.

[2] Sumit Gulwani, Oleksandr Polozov, and Rishabh Singh. Program synthesis. *Foundations and Trends in Programming Languages*, 4(1–2):1–119, 2017.

[3] Susmit Jha, Sumit Gulwani, Sanjit A. Seshia, and Ashish Tiwari. Oracle-guided component-based program synthesis. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering*, ICSE 2010, pages 215–224, 2010.

[4] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024.

[5] Oleksandr Polozov and Sumit Gulwani. Flashmeta: A framework for inductive program synthesis. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, OOPSLA 2015, pages 107–126. ACM, 2015.

[6] Armando Solar Lezama. *Program Synthesis by Sketching.* PhD thesis, EECS Department, University of California, Berkeley, December 2008.

[7] Armando Solar-Lezama. Program sketching. *International Journal on Software Tools for Technology Transfer*, 15(5–6):475–495, 2013.

[8] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit A. Seshia, and Vijay A. Saraswat. Combinatorial sketching for finite programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS 2006, pages 404–415. ACM Press, 2006.